

SPRINT 4: BIG QUERY

EJERCICIOS NIVEL 1

1. Escribe la consulta que une (JOIN) transacciones y compañías.
 - Filtra los resultados por la fecha indicada y el país "Germany".
 - Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).

Observación: Fíjate en que BigQuery lee casi toda la mesa a pesar de pedir sólo un día (Full Table Scan).

```
1 -- Ejercicio 1: El Country Manager de Alemania necesita revisar urgentemente las transacciones del día 12 de marzo de 2022 .
2
3 -- » 1. Escribe la consulta que une (JOIN) transacciones y compañías.
4 -- » 2. Filtra los resultados por la fecha indicada y el país "Germany".
5 -- » 3. Sin ejecutar la consulta, realiza un "Dry Run" (auditoría de costes).
6 -- Observación: Fíjate en que BigQuery lee casi toda la tabla a pesar de pedir sólo un día (Full Table Scan)
7
8 SELECT
9     tc.*,
10     cc.company_name,
11     cc.country
12 FROM `sprint3_silver.transactions_clean` AS tc
13 JOIN `sprint3_silver.companies_clean` AS cc
14     ON tc.business_id = cc.company_id
15 WHERE DATE(tc.timestamp) = '2022-03-12' AND cc.country = 'Germany';
16
17
```

✓ This script will process 19.58 MB when run.

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

☆ transactions_clean

Query

Chat

Open in ▾

Share ▾

Schema

Details

Preview

Table Explorer

Preview

Insights

Lineage

mode

Case insensitivefalse

Description

Labels

Primary key(s)

Tags

Storage info ?

Number of rows100,000

Total logical bytes10.17 MB

Active logical bytes10.17 MB

Long term logical bytes0 B

Current physical bytes4.58 MB

Total physical bytes22.89 MB

Active physical bytes22.89 MB

Long term physical bytes0 B

Time travel physical bytes18.31 MB

El consumo sería de 10.17 MB de procesar esa query, lo cual es similar al peso de toda la tabla transactions_clean (como se muestra en la segunda imagen).

2. Generación de datos recientes. Creación de tabla intermedia transactions_recent a partir de transactions_clean.

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

16
17
18
19 -- Ejercicio 2: generación de datos recientes
20
21 CREATE OR REPLACE TABLE `sprint3\_silver.transactions\_recent` AS
22
23 SELECT
24 * EXCEPT(timestamp), -- toma todo excepto el timestamp
25 TIMESTAMP_SUB(CURRENT_TIMESTAMP(),INTERVAL CAST(RAND() * 50 AS INT64) DAY) AS timestamp -- CURRENT da el tiempo actual, CAST RAND da un número
26 aleatorio entre 0 y 49, DAY resta ese número de días a la fecha actual
27 FROM `sprint3\_silver.transactions\_clean`;
28
29
30

✔ This script will process 19.58 MB when run.

Using on-demand processing quota

All results

Elapsed time	Statements processed	Job status
3 sec	2	✔ SUCCESS

Status	End time	SQL	Action
✔	10:55 AM [8:1]	SELECT	View results
✔	10:55 AM [19:1]	CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS	View results

Para optimizar, ahora creo una nueva tabla que contenga solo fechas recientes, de los últimos 50 días. En esta tabla se utilizan todos los campos de “transactions_clean” excepto el de timestamp, para este defino uno nuevo que se calcula cada vez de la siguiente manera: la fecha actual - un intervalo de días aleatorio.

31
32 CREATE OR REPLACE TABLE `sprint3\_gold.fact\_transactions\_optimized`
33 PARTITION BY DATE(timestamp)
34 CLUSTER BY business_id
35 AS
36 SELECT *
37 FROM `sprint3\_silver.transactions\_recent`;
38
39
40

✔ This script will process 19.58 MB when run.

Using on-demand processing quota

Query results

Chat Save results Open in

Job information

Results

Execution details

Execution graph

ⓘ This statement created a new table named fact_transactions_optimized.

Go to table

sprint3-analytics-estefaniat / Datasets / sprint3_gold / Tables / fact_transactions_optimized

☆ fact_transacti... 🔍 Query 🗨 Chat 📄 Open in ➦ 👤 Share 📄 Copy 📄 Snapshot 🗑 Delete

ⓘ This is a partitioned table. [Learn more](#)

Schema

Details

Preview

Table Explorer

Preview

Insights

Lineage

Data Profile

Data Quality

Table Type	Partitioned
Partitioned by	DAY
Partitioned on field	timestamp
Partition expiration	60 days
Partition filter	Not required

Clustered by business_id

A partir de esa tabla de fechas recientes, creo una nueva tabla optimizada particionada por la fecha y agrupada por "business_id".

3. Comprobación de rendimiento.

Paso 1: Tabla no optimizada

```
38
39
40
41 -- Ejercicio 3: Prueba del rendimiento (benchmark)
42
43 SELECT *
44 FROM `sprint3_silver.transactions_recent`
45 WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
46
47
48
49 SELECT *
50 FROM `sprint3_gold.fact_transactions_optimized`
51 WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
52
53
54
```

✓ This query will process 10.17 MB when run.

Procesaría 10.17 MB, lee toda la tabla.

Paso 2: tabla optimizada

```
40
41 -- Ejercicio 3: Prueba del rendimiento (benchmark)
42
43 SELECT *
44 FROM `sprint3_silver.transactions_recent`
45 WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
46
47
48
49 SELECT *
50 FROM `sprint3_gold.fact_transactions_optimized`
51 WHERE timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
52
53
54
```

✓ This query will process 6.02 MB when run.

Hubo una mejora/optimización de ~59% al utilizar una tabla optimizada de la capa Gold. Se redujo el procesamiento de 10.17 MB a 6 MB.

Cálculo del porcentaje:

$$(10.17-6)/(10.17)(100) = (4.17/10.17)(100) = 41 \%$$

4. Smart Caching (vistas materializadas).

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
56 -- Ejercicio 4: Vistas Materializadas
57
58 CREATE OR REPLACE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS -- vista para contar las ventas diarias
59 SELECT
60   DATE(timestamp) AS sales_date,
61   SUM(amount) AS ventas_totales_dia
62 FROM `sprint3_gold.fact_transactions_optimized`
63 WHERE declined = '0'
64 GROUP BY sales_date;
65
66 -- consultar vista:
67
68 SELECT
69   sales_date,
70   ROUND(ventas_totales_dia,2)
71 FROM `sprint3_gold.mv_daily_sales`;
```

✓ This query will process 816 B when run.
Using on-demand processing quota

← Query results Chat Save results Open in

Job information	Results	Visualization	JSON	Execution details	Execution graph
Creation time	May 26, 2026, 11:08:15 AM UTC+2				
Start time	May 26, 2026, 11:08:16 AM UTC+2				
End time	May 26, 2026, 11:08:16 AM UTC+2				
Duration	0 sec				
Bytes processed	1.81 MB				
Bytes billed	10 MB				
Slot milliseconds	1849				
Job priority	INTERACTIVE				
Use legacy SQL	false				

En este caso, interpreté las “ventas totales del día” como el número de transacciones del día.

```
65
66 -- consultar vista:
67
68 SELECT
69   sales_date,
70   ROUND(ventas_totales_dia,2)
71 FROM `sprint3_gold.mv_daily_sales`;
```

✓ This query will process 816 B when run.
Using on-demand processing quota

← Query results Chat Save results Open in

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	sales_date	f0_			
1	2026-05-15	525855.16			
2	2026-05-23	520021.88			
3	2026-05-24	513419.23			
4	2026-05-18	498757.79			
5	2026-04-28	523059.05			
6	2026-04-26	533431.2			

EJERCICIOS NIVEL 2

1. Perfilado de clientes VIP. Métricas agregadas con CTEs. Crea una CTE llamada VIP_Stats que agrupe por usuario y calcule:

El Gasto Total (SUM). La Cantidad de Transacciones (COUNT). El Ticket Medio (AVG), redondeado a 2 decimales. La Compra Máxima (MAX). Filtro: Mantiene sólo aquellos cuyo Gasto Total sea > 500. Crea la CTE con users_combined para obtener

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

los datos personales. Requisitos de Salida:Columnas: user_id, nombre_completo, email, num_compras, ticket_medio, max_compra, total_gastado. Ordenado por total_gastado descendente.

```
16 WITH VIP_Stats AS (  
17   SELECT  
18     user_id,  
19     SUM(amount) AS total_gastado,  
20     COUNT(transaction_id) AS num_compras,  
21     ROUND(AVG(amount), 2) AS ticket_medio,  
22     MAX(amount) AS max_compra  
23   FROM `sprint3-analytics-estefaniat.sprint3_gold.fact_transactions_optimized` -- lo tomo de la tabla optimizada para ahorrar MB!!  
24   GROUP BY user_id  
25   HAVING total_gastado > 500  
26 )  
27  
28 SELECT  
29   v.user_id,  
30   u.name,  
31   u.surname,  
32   u.email,  
33   v.num_compras,  
34   v.ticket_medio,  
35   v.max_compra,  
36   v.total_gastado  
37 FROM VIP_Stats AS v  
38 JOIN sprint3_silver.users_combined AS u  
39   ON v.user_id = u.user_id  
40 ORDER BY v.total_gastado DESC;  
41
```

This script will process 7.5 MB when run.

Using on-demand processing quota

←

Query results

Chat

Save results ▾

Open in ▾

↕

Job information

Results

Visualization

JSON

Execution details

Execution graph

user_id ▾	name ▾	surname ▾	email ▾	num_compras ▾	ticket_medio ▾	max_compra
289	Dxwgi	Hwcru	dxwgi.hwcru@example.com	94	441.35	€
318	Bnyr	Astuw	bnyr.astuw@example.com	91	443.39	€
455	Fwfl	Phylgnmhnd	fwfl.phylgnmhnd@example.com	66	522.6	€
417	Fqatil	Rdwcjvoa	fqatil.rdwcjvoa@example.com	78	410.46	€
185	Molly	Gilliam	donec@outlook.couk	110	275.75	€
285	Brcxprnj	Lnpna	brcxprnj.lnpna@example.com	70	417.85	€
193	Minerva	Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	€
454	Sfzzoh	Xgvfridxs	sfzzoh.xgvfridxs@example.com	81	300.67	€

2. Análisis de tendencias (Window functions sobre vistas). Crea una consulta sobre la vista materializada que genere un informe con las siguientes 4 columnas:

Fecha: La fecha de venta.

Ventas_Hoy: El total vendido aquel día.

Ventas_Ayer: El total vendido el día anterior (usando funciones de ventana).

Diff_Percentual: El porcentaje de crecimiento o decrecimiento respecto al día anterior, redondeado a 2 decimales. **Fórmula para este cálculo: (Ventas_Hoy - Ventas_Ayer) / Ventas_Ayer * 100. Ejemplo rápido:**

Ayer: 100 ventas **Hoy:** 120 ventas **Diferencia:** 20%

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
50 -- Ejercicio 2: Análisis de Tendencias (Window Functions sobre Vistas)
51
52 WITH daily_sales_lag AS (
53     SELECT
54         sales_date AS Fecha,
55         ventas_totales_dia AS Ventas_Hoy,
56         LAG(ventas_totales_dia) OVER (ORDER BY sales_date) AS Ventas_Ayer -- LAG toma la fila anterior
57     FROM `sprint3_gold.mv_daily_sales`
58 )
59
60 SELECT
61     Fecha,
62     Ventas_Hoy,
63     Ventas_Ayer,
64     ROUND(
65         SAFE_DIVIDE(Ventas_Hoy - Ventas_Ayer, Ventas_Ayer) * 100,
66         2
67     ) AS Diff_Percentual
68 FROM daily_sales_lag
69 ORDER BY Fecha;
```

✓ Query completed

Using on-demand processing quota

← Query results

Chat Save results ▾

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Fecha ▾	Ventas_Hoy ▾	Ventas_Ayer ▾	Diff_Percentual ▾	
1	2026-03-18	1016	null	null	
2	2026-03-19	1984	1016	95.28	
3	2026-03-20	2026	1984	2.12	
4	2026-03-21	1953	2026	-3.6	
5	2026-03-22	2011	1953	2.97	
6	2026-03-23	1988	2011	-1.14	

Results per page: 50 ▾ 1 – 50 of 51 |<

3. Totales Acumulados. (Running totales sobre vistas). Utilizando la vista materializada mv_daily_sales (para no recalcular agregaciones base), genera un informe con tres columnas:

- Fecha
- Ventas del Día : Redondeado a 2 decimales.
- Ventas Acumuladas YTD (year to date) : Una suma progresiva de las ventas que debe reiniciarse cada 1 de Enero . El resultado final debe estar redondeado a 2 decimales.

Pista Técnica:

Para conseguir el reinicio anual, necesitas dividir la ventana usando PARTITION BY EXTRACT(YEAR FROM data). La estructura completa sería: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
73 SELECT
74 sales_date AS Fecha,
75 ROUND(ventas_totales_dia, 2) AS Ventas_del_Dia,
76
77 ROUND(
78 SUM(ventas_totales_dia) OVER (
79 PARTITION BY EXTRACT(YEAR FROM sales_date)
80 ORDER BY sales_date
81 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
82 ),
83 2
84 ) AS Ventas_Acumuladas_YTD
85
86 FROM `sprint3_gold.mv_daily_sales`
87 ORDER BY Fecha;
```

Query completed

Using on-demand processing quota

Query results

Chat Save results Open in

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Fecha	Ventas_del_Dia	Ventas_Acumula...		
1	2026-03-18	1016.0	1016.0		
2	2026-03-19	1984.0	3000.0		
3	2026-03-20	2026.0	5026.0		
4	2026-03-21	1953.0	6979.0		
5	2026-03-22	2011.0	8990.0		
6	2026-03-23	1988.0	10978.0		
7	2026-03-24	1968.0	12946.0		

Results per page: 50 1 - 50 of 51

4. Fidelización y valor del cliente (Filtrado avanzado). Genera un listado de los usuarios que han llegado (o superado) su tercera compra, mostrando:

- Datos de usuario (user_id, nombre_completo, email) .
- La fecha y el importe exacto de la 3ª compra .
- Media de las 3 primeras : La media de gasto de sus transacciones 1, 2 y 3.

Pista Técnica:

- » Debes utilizar la función ROW_NUMBER() para establecer el orden cronológico de las compras .
- » Tienes que utilizar la cláusula QUALIFY para filtrar .
- » Reto de Optimización : Busca la estrategia de menor coste computacional. Ten en cuenta que calcular medias sobre todo el historial histórico es costoso. Trate de filtrar las filas necesarias antes de realizar la agregación final.

```
89 -- Ejercicio 4: listado de los usuarios que han superado su tercera compra
90
91 WITH ordered_transactions AS ( -- se crea una tabla temporal con las 3 primeras compras de cada usuario
92 SELECT
93 user_id,
94 transaction_id,
95 timestamp,
96 amount,
97
98 ROW_NUMBER() OVER (
99 PARTITION BY user_id
100 ORDER BY timestamp
101 ) AS purchase_number
102
103 FROM `sprint3-analytics-estefaniat.sprint3_silver.transactions_clean`
104 QUALIFY purchase_number <= 3
105 ),
106
107 first_three_stats AS ( -- calcula la media solo sobre esas 3 compras, no sobre todo el historial
108 SELECT
109 user_id,
110 AVG(amount) AS media_3_primeras
111 FROM ordered_transactions
112 GROUP BY user_id
113 HAVING COUNT(*) = 3
114 ),
115
116 third_purchase AS (
117 SELECT
118 user_id,
119 timestamp AS fecha_tercera_compra,
120 amount AS importe_tercera_compra
121 FROM ordered_transactions
122 WHERE purchase_number = 3
123 ),
124
```


Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
125 users AS (  
126   SELECT  
127     user_id,  
128     CONCAT(name, ' ', surname) AS nombre_completo,  
129     email  
130   FROM `sprint3-analytics-estefaniat.sprint3_silver.users_combined`  
131 )  
132  
133 SELECT  
134   u.user_id,  
135   u.nombre_completo,  
136   u.email,  
137   t.fecha_tercera_compra,  
138   t.importe_tercera_compra,  
139   ROUND(s.media_3_primeras, 2) AS media_3_primeras  
140 FROM third_purchase AS t  
141 JOIN first_three_stats AS s  
142   ON t.user_id = s.user_id  
143 JOIN users AS u  
144   ON t.user_id = u.user_id  
145 ORDER BY t.fecha_tercera_compra;  
146
```

✓ This script will process 7.5 MB when run.

Using on-demand processing quota

Query results							Chat	Save results ▾	Open in ▾	↕
Job information							Results			
Visualization							JSON			
Execution details							Execution graph			
Row	user_id	nombre_completo	email	fecha_tercera_compra	importe_tercera...	media_3_primeras				
1	1726	Fxhuor Hlgqfldp	fxhuor.hlgqfldp@example.com	2015-01-25 15:12:10 UTC	414.22	231.0				
2	525	Qorntk Puahqoxq	qorntk.puahqoxq@example.com	2015-01-28 11:34:36 UTC	81.33	197.07				
3	2546	Beqljy Dcvyfvxy	beqljy.dcvyfvxy@example.com	2015-02-01 06:59:42 UTC	26.51	129.27				

EJERCICIOS NIVEL 3

1. Desanimiento y allanamiento de datos.

```
6  
7 SELECT product_ids  
8 FROM `sprint3_silver.transactions_clean`
```

✓ Query completed

Using on-demand processing quota

Query results

Job information		Results	Visualization	JSON	Execut
Row	product_ids				
1	55				
2	46, 48, 24				
3	14, 39, 65, 27				
4	61, 18, 17, 58, 42				
5	56				
6	12, 7, 87, 96				
7	19, 25, 14				

Así se encuentran los `product_ids` en `transactions_clean`, en strings dentro de una sola celda...

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS

SELECT
  t.transaction_id,
  t.timestamp,
  t.amount AS total_ticket,
  TRIM(product_id) AS product_sku, -- product_id de transactions_clean tiene espacios, TRIM los quita
  p.name AS product_name,
  p.price AS product_price

FROM `sprint3_silver.transactions_clean` AS t
CROSS JOIN UNNEST(SPLIT(t.product_ids, ',')) AS product_id
JOIN `sprint3_silver.products_clean` AS p
  ON SAFE_CAST(TRIM(product_id) AS INT64) = SAFE_CAST(p.product_id AS INT64);
```

Query completed

Query on-demand processing quota

Query results

Information Results Execution details Execution graph

This statement replaced the table named dim_transactions_flat.

Con la función *SPLIT*, convierto los strings de *product_ids* en arrays tomando en cuenta la “,” como separador de elementos. *UNNEST* convierte el array en filas por cada elemento y renombro esto como “*product_id*”. Cross join me permite unir estas nuevas filas con todas las de *transactions_clean* y luego a esto agrego la tabla *products_clean*.

Utilizo *SAFE_CAST* con *TRIM* para quitar cualquier espacio de *product_id* y asegurarme que es un *INT* compatible con *product_id* de *products_clean*. Entiendo que esto debería realizarse en la “depuración” de los datos en la capa *Bronce* para evitar todas estas conversiones.

sprint3-analytics-estefaniat / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transactions_flat 🔍 Query 💬 Chat 📄 Open in ▾ 👤 Share ▾ 📄 Copy 📄 Snapshot 🗑 Delete 📄 Export

Schema	Details	Preview	Table Explorer	Preview	Insights	Lineage	Data Profile	Data Quality
Row	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price		
1	EA7996CC-0D24-4089-82FD-29...	2024-12-01 23:05:35 UTC	14	Direwolf	147.53			
2	E1538933-0682-4498-9C16-3E...	2017-04-04 04:48:57 UTC	14	Direwolf	147.53			
3	DD903230-8925-45FF-B9D3-1C...	2022-06-06 17:46:57 UTC	14	Direwolf	147.53			
4	6150642A-EA77-4537-A0C0-AC...	2016-03-07 21:02:10 UTC	14	Direwolf	147.53			
5	3779CB05-1B6B-4711-82D4-8F...	2017-12-13 09:58:11 UTC	14	Direwolf	147.53			
6	41A9628D-BC3A-42A7-86E6-5E...	2024-05-26 08:37:04 UTC	14	Direwolf	147.53			
7	6E2520B3-B912-4261-A21F-4A...	2020-10-22 21:36:40 UTC	14	Direwolf	147.53			
8	88277578-2CF3-4036-8B9B-10...	2021-01-09 09:19:47 UTC	14	Direwolf	147.53			
9	477D10C2-A9AC-4FBB-9B60-14...	2017-08-31 17:00:52 UTC	14	Direwolf	147.53			
10	4C75930F-781D-4124-8632-990...	2021-08-26 11:54:29 UTC	14	Direwolf	147.53			
11	8BBFE9A-C45A-4DE9-B19E-6C...	2020-08-19 20:59:12 UTC	14	Direwolf	147.53			
12	E3276321-BE12-4894-AFBE-28...	2023-01-09 21:28:37 UTC	14	Direwolf	147.53			
13	36CDB705-F6C6-43BE-AE0C-21...	2016-08-19 23:31:13 UTC	14	Direwolf	147.53			
14	9CCAA4A6-3FBC-488D-A927-B...	2018-07-02 03:50:20 UTC	14	Direwolf	147.53			
15	86657B7E-4B9D-4820-AA27-8C...	2021-04-25 17:20:07 UTC	14	Direwolf	147.53			
16	41E72331-612A-4FF1-ACDE-2F...	2019-04-19 22:14:25 UTC	14	Direwolf	147.53			

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

SchemaDetailsPreviewTable ExplorerPreviewInsightsLineageData ProfileData Quality

Generate table and column descriptions with Gemini. Ensure a profile scan to [ground](#) the descriptions.

DismissGenerate

Filter Enter property name or value

	Field name	Type	Mode	Description	Key	Collation	Default Value	Policy Tags	Data Policies
☐	transaction_id	STRING	NULLABLE	-	-	-	-	-	-
☐	timestamp	TIMESTAMP	NULLABLE	-	-	-	-	-	-
☐	total_ticket	FLOAT	NULLABLE	-	-	-	-	-	-
☐	product_sku	STRING	NULLABLE	-	-	-	-	-	-
☐	product_name	STRING	NULLABLE	-	-	-	-	-	-
☐	product_price	FLOAT	NULLABLE	-	-	-	-	-	-

2. Ranking de ventas / agregación simple.

21 -- Ejercicio 2: Ranking de ventas

22

23 SELECT

24 product_name,

25 COUNT(*) AS unidades_vendidas

26 FROM `sprint3\_gold\_dim\_transactions\_flat`

27 GROUP BY product_name

28 ORDER BY unidades_vendidas DESC

29 LIMIT 5;

30

✔ This script will process 6.1 MB when run.

Using on-demand processing quota

← Query results

ChatSave resultsOpen in

Job informationResultsVisualizationJSONExecution detailsExecution graph

Row	product_name	unidades_vendidas
1	Winterfell	10071
2	the duel	10059
3	dooku solo	10022
4	Karstark Dorne	7694
5	skywalker ewok	7607

Gracias a la tabla “aplanada” ahora se pueden realizar queries muy sencillos para ver cada producto por separado.

3. Automatización del pipeline y automatización.

Por: Estefanía Thompson Valdivia
P2P: Jorge Aguilar

```
37
38 CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64)
39 RETURNS FLOAT64
40 AS (
41   ROUND(amount * 1.21, 2)
42 );
43
44 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
45
46 SELECT
47   t.transaction_id,
48   t.timestamp,
49   SAFE_CAST(t.amount AS FLOAT64) AS total_ticket,
50   TRIM(product_id) AS product_sku,
51   p.name AS product_name,
52   p.price AS product_price,
53   `sprint3_gold.calculate_tax`(SAFE_CAST(p.price AS FLOAT64)) AS product_price_tax_inc -- se llama a la función
54
55 FROM `sprint3_silver.transactions_clean` AS t
56 CROSS JOIN UNNEST(SPLIT(t.product_ids, ',')) AS product_id
57 JOIN `sprint3_silver.products_clean` AS p
58   ON SAFE_CAST(TRIM(product_id) AS INT64) = SAFE_CAST(p.product_id AS INT64);
59
```

Export as scheduled query

 Settings valid.

Details and schedule

Name for scheduled query *
refresh_dim_transactions_flat

Schedule options

Repeat frequency *
Days

At *
07:00 UTC

☒ Start now ☐ Start at set time

Start date and run time
5/12/26, 12:55PM CEST

☒ End never ☐ Schedule end time

End date CET

 Scheduled to run every day at 07:00 UTC starting now.

Destination for query results

☐ Set a destination table for query results

Save

Cancel

**Me falta agregar enlace a Data Studio.*